

Chapter 3: AWS CLI, SDK & Access Methods

Phase: Cloud Foundations | **Difficulty:** Beginner | **SAA-C03 Domain:** Design Secure Architectures

Table of Contents

1. [Chapter Overview](#)
 2. [Prerequisites](#)
 3. [Topic 1 — AWS CLI v2: Installation, Configuration & Command Structure](#)
 4. [Topic 2 — AWS CloudShell: Browser-Based CLI](#)
 5. [Topic 3 — Access Keys vs. IAM Roles for Programmatic Access](#)
 6. [Topic 4 — Credential Provider Chain and Precedence Order](#)
 7. [Topic 5 — Named Profiles and Configuration in ~/.aws/config](#)
 8. [Topic 6 — AWS SDK Overview: Boto3, Node.js, Java](#)
 9. [Hands-On Labs](#)
 - [Lab 1 — Install and Configure AWS CLI v2](#)
 - [Lab 2 — Run Core CLI Commands](#)
 - [Lab 3 — Use AWS CloudShell](#)
 - [Lab 4 — Configure Multiple Named CLI Profiles](#)
 10. [Real-World Scenarios](#)
 11. [Key Facts & Exam Cheat Sheet](#)
 12. [Certification-Style Practice Questions](#)
 13. [Interview Questions](#)
 14. [Chapter Summary](#)
-

Chapter Overview

In the previous two chapters you learned what AWS is and how to secure access to it using IAM. This chapter answers a critical practical question: **how do developers, administrators, and automated systems actually talk to AWS?**

The AWS Management Console (the website) is great for exploration, but it is impractical for automation, scripting, and repeatable deployments. AWS provides three programmatic access methods:

Access Method	Best For
AWS CLI	Shell scripts, one-off commands, CI/CD pipelines
AWS CloudShell	Quick browser-based commands with zero setup
AWS SDKs	Application code in Python, Node.js, Java, etc.

All three methods share the same underlying mechanism: they call **AWS APIs** using credentials (Access Keys or IAM Roles). Understanding how credentials are found, ranked, and used is essential both for building applications and for passing the SAA-C03 exam.

By the end of this chapter you will be able to:

- Install and configure the AWS CLI v2 on Windows, macOS, and Linux
- Execute real AWS commands from your terminal
- Use AWS CloudShell without any local installation
- Explain the security difference between Access Keys and IAM Roles
- Understand the exact order AWS uses to find credentials (the credential provider chain)
- Create and switch between multiple named CLI profiles
- Write basic AWS SDK code using Python (Boto3) and Node.js

Prerequisites

- An AWS account (free tier is sufficient for all labs)
- Chapter 1 (AWS Fundamentals) and Chapter 2 (IAM Essentials) completed
- Basic familiarity with a terminal / command prompt
- For SDK sections: basic Python or JavaScript knowledge is helpful but not required

Topic 1 — AWS CLI v2: Installation, Configuration & Command Structure

What Is the AWS CLI?

The **AWS Command Line Interface (CLI)** is an open-source tool that lets you control AWS services directly from your terminal. Think of it as a text-based version of the AWS Management Console. Instead of clicking buttons, you type commands.

Real-world analogy: Imagine you need to copy 500 files from one S3 bucket to another. Doing this through the console would require hundreds of clicks. With the CLI, it is a single command:

```
aws s3 sync s3://source-bucket s3://destination-bucket
```

CLI v1 vs. CLI v2

AWS has two major versions. **Always use CLI v2** for new setups.

Feature	CLI v1	CLI v2
Installation	pip (Python package)	Standalone installer (no Python dep.)
AWS SSO	Limited	Full native support
Output paging	No	Yes (built-in pager)
Configuration wizard	No	Yes (aws configure)
Performance	Good	Better
Support status	Maintenance only	Actively developed

Installing AWS CLI v2

On Windows

1. Download the official MSI installer
from: <https://awscli.amazonaws.com/AWSCLIV2.msi>
2. Double-click the .msi file and follow the wizard (Next → Next → Install).
3. Open a new **Command Prompt** or **PowerShell** window and verify:

```
aws --version  
# Expected output: aws-cli/2.x.x Python/3.x.x Windows/...
```

On macOS

```
# Option 1: Official PKG installer (recommended)  
curl "https://awscli.amazonaws.com/AWSCLIV2.pkg" -o "AWSCLIV2.pkg"  
sudo installer -pkg AWSCLIV2.pkg -target /  
  
# Option 2: Homebrew  
brew install awscli  
  
# Verify  
aws --version
```

On Linux (x86_64)

```
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o  
"awscliv2.zip"  
unzip awscliv2.zip  
sudo ./aws/install  
  
# Verify  
aws --version
```

Important: Do NOT install via `pip install awscli` — that installs the old v1.
Always use the official installer for v2.

Basic Configuration

Before using the CLI, you need to tell it which AWS account to connect to and how to authenticate. The simplest way is the **configuration wizard**:

```
aws configure
```

You will be prompted for four values:

```
AWS Access Key ID [None]: AKIAIOSFODNN7EXAMPLE
AWS Secret Access Key [None]: wJalrXUtnFEMI/K7MDENG/bPxrFiCYEXAMPLEKEY
Default region name [None]: us-east-1
Default output format [None]: json
```

Let's understand each field:

AWS Access Key ID — A 20-character uppercase identifier beginning with AKIA. Think of it as your "username" for programmatic access.

AWS Secret Access Key — A 40-character secret string. Think of it as your "password". Never share this. Never commit it to Git.

Default region name — The AWS Region where your commands execute by default. Use `us-east-1` (US East N. Virginia) as it has the most services available and is a common exam default.

Default output format — How CLI responses are displayed:

- `json` — structured data, best for parsing by scripts
- `table` — human-readable, formatted columns
- `text` — plain text, tab-separated, great for shell scripting with `grep/awk`

Where Configuration Is Stored

After running `aws configure`, two files are created:

```
~/.aws/credentials (on Windows: C:\Users\USERNAME\.aws\credentials)
~/.aws/config      (on Windows: C:\Users\USERNAME\.aws\config)
```

~/.aws/credentials — stores sensitive key material:

```
[default]
aws_access_key_id = AKIAIOSFODNN7EXAMPLE
aws_secret_access_key = wJalrXUtnFEMI/K7MDENG/bPxrFiCYEXAMPLEKEY
```

~/.aws/config — stores non-sensitive settings:

```
[default]
region = us-east-1
output = json
```

Security Note: Both files should have restricted permissions. On Linux/macOS, AWS CLI automatically sets them to `600` (owner read/write only). Never store these files in a public repository.

CLI Command Structure

Every AWS CLI command follows this pattern:

```
aws <service> <operation> [--parameter value] [--parameter value] ...
```

Breaking this down with examples:

```
# aws <service>      <operation>      <parameters>
aws  ec2             describe-instances
aws  s3              ls                s3://my-bucket
aws  iam             list-users
aws  s3api            put-bucket-policy --bucket my-bucket --policy
file://policy.json
```

Common global options (work with any command):

Option	Description	Example
--region	Override the default region	--region eu-west-1
--output	Override the default output format	--output table
--profile	Use a named profile	--profile prod
--query	Filter output using JMESPath	--query 'Reservations[*].Instances[*].InstanceId'
--dry-run	Test if you have permission without executing	--dry-run
--no-cli-pager	Disable the built-in pager for scripting	--no-cli-pager

Using --query to Filter Output

The --query parameter uses **JMESPath** syntax to extract specific fields from JSON output. This is very useful in scripts.

```
# List only the InstanceId and State of all EC2 instances
aws ec2 describe-instances \
  --query 'Reservations[*].Instances[*].{ID:InstanceId,State:State.Name}' \
  --output table
```

Output:

```
-----+-----+-----+
|           DescribeInstances           |
+-----+-----+-----+
|           ID           |      State      |
+-----+-----+-----+
| i-0abc123def456789    |    running    |
| i-0def456abc123789    |    stopped    |
+-----+-----+-----+
```

Pagination

Many AWS commands return large result sets. The CLI handles pagination automatically for most

commands:

```
# This automatically fetches ALL pages
aws s3api list-objects-v2 --bucket my-large-bucket
```

For manual pagination control:

```
# Get only the first 10 results
aws ec2 describe-instances --max-items 10

# Get the next page using the NextToken
aws ec2 describe-instances --max-items 10 --starting-token <NextToken>
```

Getting Help

The CLI has built-in documentation:

```
# General help
aws help

# Service-level help
aws s3 help

# Command-level help
aws s3 cp help

# List all available commands for a service
aws ec2 help | grep "describe-"
```

Topic 2 — AWS CloudShell: Browser-Based CLI

What Is AWS CloudShell?

AWS CloudShell is a browser-based shell environment provided directly inside the AWS Management Console. It comes with the AWS CLI pre-installed, pre-authenticated, and pre-configured — you do not need to install anything or set up credentials.

Real-world analogy: Imagine you are travelling and need to run an AWS command from a hotel computer. You cannot install software on it. CloudShell solves this problem — you just open a browser, log into AWS, and start typing commands.

Key Features

Feature	Detail
Authentication	Automatically uses your console session credentials
Pre-installed tools	AWS CLI v2, Python 3, Node.js, git, pip, npm, jq, vim, nano
Storage	1 GB of persistent storage per Region (survives session restarts)
Compute	Runs in a managed Linux environment (Amazon Linux 2)

Feature	Detail
Regions	Available in most AWS Regions
Cost	Free — no additional charges
Timeout	Session times out after 20–30 minutes of inactivity

How to Open CloudShell

1. Log into the **AWS Management Console**
2. Click the **CloudShell icon** in the top navigation bar (looks like a terminal >_)
3. A terminal window opens at the bottom of the page
4. Wait 10–20 seconds for the environment to initialize

Alternatively, search for "CloudShell" in the services search bar.

What You Can Do in CloudShell

Because CloudShell is pre-authenticated with your console credentials, you can immediately run commands:

```
# Verify who you are
aws sts get-caller-identity

# List your S3 buckets
aws s3 ls

# List EC2 instances in the current region
aws ec2 describe-instances --output table

# Change region for a single command
aws s3 ls --region eu-west-1

# Use Python (Boto3 is pre-installed)
python3 -c "import boto3; print(boto3.__version__)"
```

Persistent Storage in CloudShell

CloudShell provides 1 GB of persistent storage in your home directory (/home/cloudshell-user). Files you save here persist across sessions:

```
# Create a script and save it
cat > my-script.sh << 'EOF'
#!/bin/bash
echo "My EC2 instances:"
aws ec2 describe-instances \
  --query 'Reservations[*].Instances[*].{ID:InstanceId,Type:InstanceType}' \
  --output table
EOF

chmod +x my-script.sh
./my-script.sh
```

Next time you open CloudShell (even days later), `my-script.sh` will still be there.

Limitations of CloudShell

- Cannot be used for long-running processes (session timeout)
- Not suitable for high-volume data transfers
- Cannot run GUI applications
- Storage is Region-specific (1 GB in us-east-1 is separate from 1 GB in eu-west-1)
- Maximum 10 concurrent shells per Region

CloudShell vs. Local CLI — When to Use Which

Scenario	Use CloudShell	Use Local CLI
Quick one-off command		
Working on someone else's computer		
CI/CD pipeline		
Long-running scripts		
Working offline		
Demo / training		
Secure environment (no key files)		

Topic 3 — Access Keys vs. IAM Roles for Programmatic Access

The Core Problem: Proving Identity to AWS APIs

When a CLI command or SDK call reaches an AWS API endpoint, the first thing AWS asks is: **"Who are you, and are you allowed to do this?"**

There are two ways to prove your identity programmatically:

1. **Access Keys** — a static username/password pair you manage yourself
2. **IAM Roles** — temporary credentials that AWS automatically issues and rotates

Access Keys: How They Work

Access Keys consist of two parts:

```
Access Key ID:      AKIAIOSFODNN7EXAMPLE      (20 chars, starts with AKIA)
Secret Access Key:  wJalrXUtnFEMI/K7MDENG... (40 chars, random)
```

When you run a CLI command or make an SDK call, the tool uses these keys to cryptographically sign the HTTP request using **AWS Signature Version 4 (SigV4)**. AWS verifies the signature to confirm the request is authentic.

Creating Access Keys:

1. Go to **IAM** → **Users** → [your user] → **Security credentials**
2. Click **Create access key**
3. Choose the use case (CLI, local code, etc.)
4. Download the .csv file — **this is the only time you can see the secret key**

Critical limitation: The Secret Access Key is shown only once at creation. If you lose it, you must delete the key pair and create a new one.

Problems with Access Keys

Access Keys seem simple, but they introduce serious security risks if mismanaged:

Problem 1: Long-lived credentials Access keys are valid indefinitely until you manually rotate or delete them. If stolen, an attacker can use them for months.

Problem 2: Accidental exposure Developers accidentally commit access keys to GitHub every day. AWS monitors public repositories and notifies you (and may automatically deactivate the key), but damage can be done in minutes.

```
# This has happened to real companies:
git add .
git commit -m "added AWS config"
git push origin main
# 🤖 credentials/config.py contained hardcoded keys – now public
```

Problem 3: Distribution complexity If multiple systems need access, you must create, distribute, and track multiple key pairs.

Problem 4: Rotation burden Security best practices require rotating keys every 90 days. This is manual, error-prone, and often neglected.

IAM Roles: A Better Way

An **IAM Role** is an identity with permissions, but unlike a user, it has **no permanent credentials**. Instead:

1. An entity (EC2 instance, Lambda function, CI/CD pipeline) **assumes** the role
2. AWS STS (Security Token Service) issues **temporary credentials** (Access Key ID + Secret Key + **Session Token**)
3. These credentials expire automatically after 15 minutes to 12 hours
4. If the credentials leak, they are useless after expiry

How it works with EC2:

```
EC2 Instance (with IAM Role attached)
|
|— App requests credentials from Instance Metadata Service
http://169.254.169.254/latest/meta-data/iam/security-credentials/RoleName
|
|— Gets back: {
```

```

(temporary) "AccessKeyId": "ASIAIOSFODNN7EXAMPLE",      ← starts with ASIA
temp creds) "SecretAccessKey": "...",
expiry      "Token": "AQoXnyc4lck4w...",              ← session token (unique to
              "Expiration": "2024-01-15T12:00:00Z"      ← auto-rotated before
              }

```

The AWS CLI and SDKs handle this automatically. When running on an EC2 instance with a role, your code does not need any key configuration at all.

Security Comparison

Security Factor	Access Keys	IAM Roles
Credential lifetime	Permanent (until deleted)	Temporary (15 min – 12 hours)
Rotation	Manual, often neglected	Automatic
Risk of exposure	High (stored in files)	Low (never stored; delivered on-demand)
Works on EC2/Lambda	(but bad practice)	(best practice)
Works on a developer laptop		△ (via <code>aws sts assume-role</code> or SSO)
Recommended for	Local dev, CI/CD (with caution)	All AWS compute resources

The Golden Rules

Rule 1: NEVER embed access keys directly in application code. **Rule 2:** NEVER store access keys in environment variables on AWS compute resources. **Rule 3:** Use IAM Roles for any AWS service that runs code (EC2, Lambda, ECS, etc.). **Rule 4:** Use access keys ONLY for local CLI use and rotate them regularly. **Rule 5:** Enable MFA on the IAM user that owns the access keys.

Temporary Security Credentials (STS)

When an entity assumes a role, AWS STS returns **three-part temporary credentials**:

```

{
  "Credentials": {
    "AccessKeyId": "ASIAIOSFODNN7EXAMPLE",
    "SecretAccessKey": "wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY",
    "SessionToken": "AQoXnyc4lck4w...<very long string>",
    "Expiration": "2024-01-15T12:00:00Z"
  }
}

```

Notice that temporary keys start with **ASIA** instead of **AKIA**. This is how you can tell temporary from permanent credentials.

The `SessionToken` is a required third piece — temporary credentials will not work without it.

Topic 4 — Credential Provider Chain and Precedence Order

The Problem the Chain Solves

Consider this scenario: you have Access Keys configured in `~/.aws/credentials`, but you are also running on an EC2 instance with an IAM Role. You have also set the `AWS_ACCESS_KEY_ID` environment variable for testing. Which credentials does the CLI actually use?

AWS solves this with a defined, ordered **credential provider chain** — a sequence of locations it checks in order, using the first set of valid credentials it finds.

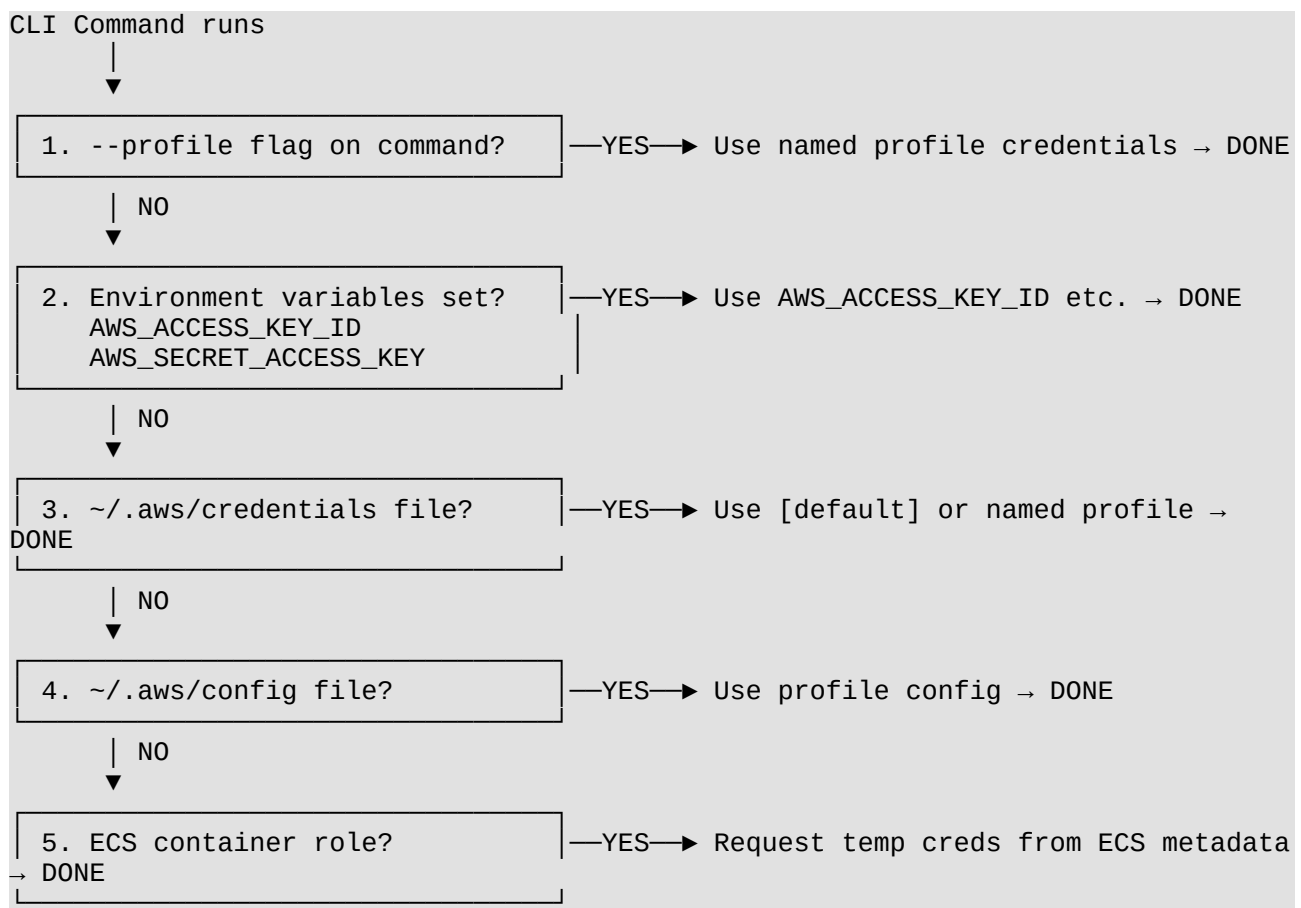
The Complete Credential Provider Chain

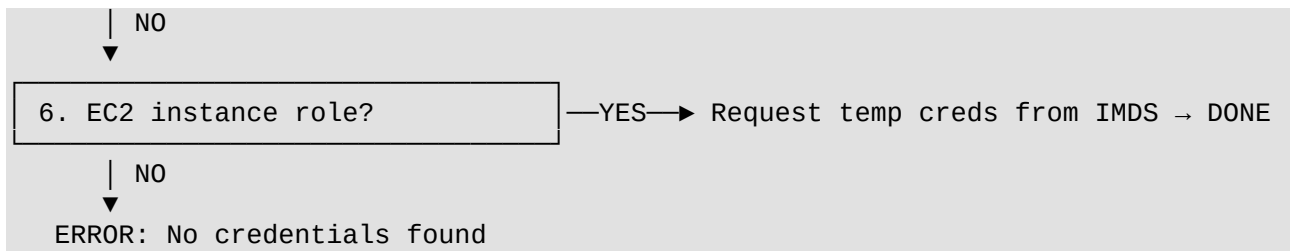
The CLI and all SDKs check credential sources in this **exact order**:

1. Command-line options (`--profile`, directly passed credentials)
2. Environment variables (`AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`, `AWS_SESSION_TOKEN`)
3. AWS CLI credentials file (`~/.aws/credentials`)
4. AWS CLI config file (`~/.aws/config`)
5. Container credentials (ECS task role via `AWS_CONTAINER_CREDENTIALS_RELATIVE_URI`)
6. EC2 instance profile (IAM Role attached to EC2 via Instance Metadata Service)

AWS checks each source from top to bottom and **stops at the first one that provides valid credentials**.

Visualising the Chain





Environment Variables for Credentials

Setting credentials via environment variables is useful for CI/CD pipelines and temporary overrides. The variables are:

```
# Required
export AWS_ACCESS_KEY_ID="AKIAIOSFODNN7EXAMPLE"
export AWS_SECRET_ACCESS_KEY="wJalrXUtnFEMI/K7MDENG/bPxrFiCYEXAMPLEKEY"

# Required only for temporary (STS) credentials
export AWS_SESSION_TOKEN="AQoXnyc4lcK4w..."

# Optional overrides
export AWS_DEFAULT_REGION="eu-west-1"
export AWS_DEFAULT_OUTPUT="json"
export AWS_PROFILE="production"
```

Important: Environment variables override the `~/.aws/credentials` file. This is useful when you want to temporarily use different credentials without modifying the credentials file.

```
# This uses credentials from ~/.aws/credentials [default]
aws s3 ls

# This uses the environment variable (overrides the file)
AWS_ACCESS_KEY_ID=AKIA... AWS_SECRET_ACCESS_KEY=... aws s3 ls

# Clear environment variables to fall back to the file
unset AWS_ACCESS_KEY_ID
unset AWS_SECRET_ACCESS_KEY
```

Why the Chain Matters for the Exam

The SAA-C03 exam regularly tests your understanding of the credential chain with scenario questions like:

"A developer has `AWS_ACCESS_KEY_ID` set in their environment. They also have a `[default]` profile in `~/.aws/credentials`. They run `aws s3 ls --profile dev`. Which credentials are used?"

Answer: The `--profile dev` credentials, because a named profile specified on the command line takes highest priority.

"An EC2 instance has an IAM Role attached. A developer accidentally left `AWS_ACCESS_KEY_ID` set in the environment variables on the instance. Which credentials are used by an application running on that instance?"

Answer: The environment variable credentials (position 2 in the chain), because they are checked before the instance profile (position 6). This is why you should never set credential environment

variables on EC2 instances — they override the preferred IAM Role mechanism.

Verifying Which Credentials Are in Use

Always verify which identity you are currently using:

```
aws sts get-caller-identity
```

Output:

```
{
  "UserId": "AIDAIOSFODNN7EXAMPLE",
  "Account": "123456789012",
  "Arn": "arn:aws:iam::123456789012:user/john.doe"
}
```

This command is invaluable for debugging credential issues. If you are using a role, the ARN will show `assumed-role` instead of `user`.

Topic 5 — Named Profiles and Configuration in `~/.aws/config`

Why Named Profiles Exist

Most professionals work with **multiple AWS accounts** — for example, separate accounts for development, staging, and production. Named profiles let you store and easily switch between configurations for each account.

Real-world scenario: You work at a company with:

- Personal sandbox account (for learning)
- Development account (shared with the dev team)
- Production account (limited, audited access)

Instead of editing `~/.aws/credentials` every time you switch contexts, you create a named profile for each.

Creating Named Profiles

Method 1: Interactive wizard

```
# Creates a profile named "dev"
aws configure --profile dev

# Creates a profile named "prod"
aws configure --profile prod
```

Method 2: Manually editing the files

`~/.aws/credentials`:

```
[default]
aws_access_key_id = AKIAIOSFODNN7EXAMPLE
aws_secret_access_key = wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY
```

```
[dev]
aws_access_key_id = AKIAI44QH8DHBEXAMPLE
aws_secret_access_key = je7MtGbClwBF/2Zp9Utk/h3yCo8nvbEXAMPLEKEY

[prod]
aws_access_key_id = AKIAI44QH8DHBEXAMPLE2
aws_secret_access_key = je7MtGbClwBF/2Zp9Utk/h3yCo8nvbEXAMPLEKEY2
```

~/.aws/config:

```
[default]
region = us-east-1
output = json

[profile dev]
region = eu-west-1
output = table

[profile prod]
region = us-east-1
output = json
```

Important syntax difference: In ~/.aws/credentials, named profiles use [profile-name]. In ~/.aws/config, named profiles use [profile profile-name] (the word "profile" is prefixed). The [default] section is the exception — it uses [default] in both files.

Using Named Profiles

```
# Use the default profile
aws s3 ls

# Use the "dev" profile for one command
aws s3 ls --profile dev

# Use the "prod" profile for one command
aws ec2 describe-instances --profile prod

# Set an environment variable to change the default profile for the session
export AWS_PROFILE=dev
aws s3 ls      # now uses "dev" profile
aws ec2 describe-instances  # also uses "dev" profile

# Reset back to default
unset AWS_PROFILE
```

Profile for Assuming Roles

Named profiles can also automatically assume an IAM Role. This is useful for cross-account access:

```
# ~/.aws/config
[profile prod-admin]
role_arn = arn:aws:iam::PROD_ACCOUNT_ID:role/AdminRole
source_profile = default
region = us-east-1
```

When you use `--profile prod-admin`, the CLI automatically:

1. Uses the default profile's credentials
2. Calls `sts:AssumeRole` with the specified role ARN
3. Uses the resulting temporary credentials for the command

```
# This automatically assumes the role
aws s3 ls --profile prod-admin
```

Advanced Configuration Options

`~/.aws/config` supports many additional settings per profile:

```
[profile dev]
region = eu-west-1
output = json
# Maximum number of CLI retry attempts
max_attempts = 5
# Retry mode (legacy, standard, adaptive)
retry_mode = standard
# CLI pager (disable with empty string)
cli_pager =
# Enable colored output
cli_auto_prompt = on
# Duration for assumed role credentials (in seconds)
duration_seconds = 3600
```

Listing and Verifying Profiles

```
# List all configured profiles
aws configure list-profiles

# Show what configuration a specific profile has
aws configure list --profile dev

# Output:
#      Name                               Value                               Type    Location
#      ----                               -
#      profile                             dev                                manual   --profile
#      access_key       *****MPLE shared-credentials-file
#      secret_key       *****EKEY  shared-credentials-file
#      region           eu-west-1      config-file    ~/.aws/config
```

Setting and Getting Individual Config Values

```
# Set a single config value
aws configure set region ap-southeast-1 --profile staging
aws configure set output table --profile staging

# Get a single config value
aws configure get region --profile staging
# Output: ap-southeast-1
```

Topic 6 — AWS SDK Overview: Boto3, Node.js, Java

What Is an AWS SDK?

An **SDK (Software Development Kit)** is a library for your programming language that wraps AWS API calls into native functions and objects. Instead of manually crafting HTTP requests with SigV4 signatures, you call simple methods like `s3.listBuckets()` or `ec2.describeInstances()`.

SDKs handle:

- Authentication and credential loading (same credential provider chain as CLI)
- Request signing (SigV4)
- Automatic retries with exponential backoff
- Response parsing (JSON → native objects)
- Pagination helpers
- Error handling

AWS SDK for Python — Boto3

Boto3 is the official AWS SDK for Python. It is the most widely used AWS SDK and is pre-installed in AWS Lambda's Python runtime and in CloudShell.

Installation:

```
pip install boto3
```

Basic structure — two types of objects:

Object	Description	Example
Client	Low-level, maps 1:1 to AWS API operations	<code>boto3.client('s3')</code>
Resource	High-level, object-oriented abstraction	<code>boto3.resource('s3')</code>

Use **Client** when you need full control or the Resource abstraction is not available. Use **Resource** for common operations — it is simpler to read and write.

Example 1: List S3 Buckets (Client)

```
import boto3

# Create a client – uses the same credential chain as CLI
s3_client = boto3.client('s3', region_name='us-east-1')

# Call the API
response = s3_client.list_buckets()

# response is a dictionary matching the AWS API JSON structure
for bucket in response['Buckets']:
    print(f"Bucket: {bucket['Name']}, Created: {bucket['CreationDate']}")
```

Example 2: Upload a file to S3 (Resource)

```
import boto3

s3 = boto3.resource('s3')
bucket = s3.Bucket('my-bucket-name')

# Upload a file
bucket.upload_file('local-file.txt', 'remote-key.txt')
print("Upload complete!")
```

Example 3: Launch an EC2 instance

```
import boto3

ec2 = boto3.resource('ec2', region_name='us-east-1')

instances = ec2.create_instances(
    ImageId='ami-0c55b159cbfafa1f0',      # Amazon Linux 2 AMI
    MinCount=1,
    MaxCount=1,
    InstanceType='t2.micro',
    KeyName='my-key-pair',
    SecurityGroupIds=['sg-0abc12345def67890']
)

print(f"Launched instance: {instances[0].id}")
```

Example 4: Using a named profile in Boto3

```
import boto3

# Create a session using a specific profile
session = boto3.Session(profile_name='dev')
s3 = session.client('s3')
response = s3.list_buckets()
```

Example 5: Error handling

```
import boto3
from botocore.exceptions import ClientError

s3 = boto3.client('s3')

try:
    s3.get_object(Bucket='my-bucket', Key='non-existent-file.txt')
except ClientError as e:
    error_code = e.response['Error']['Code']
    if error_code == 'NoSuchKey':
        print("The file does not exist.")
    elif error_code == 'AccessDenied':
        print("You do not have permission to access this file.")
    else:
        print(f"Unexpected error: {e}")
```

AWS SDK for JavaScript (Node.js) — AWS SDK v3

The AWS SDK for JavaScript v3 is the modern version, released in 2020. It uses a modular design — you import only the services you need, reducing bundle size.

Installation:

```
# Install specific service clients (v3 is modular)
npm install @aws-sdk/client-s3
npm install @aws-sdk/client-ec2
npm install @aws-sdk/client-iam
```

Example 1: List S3 Buckets

```
const { S3Client, ListBucketsCommand } = require('@aws-sdk/client-s3');

// Create client – uses same credential chain as CLI
const client = new S3Client({ region: 'us-east-1' });

async function listBuckets() {
  try {
    const command = new ListBucketsCommand({});
    const response = await client.send(command);
    response.Buckets.forEach(bucket => {
      console.log(`Bucket: ${bucket.Name}`);
    });
  } catch (error) {
    console.error('Error:', error);
  }
}

listBuckets();
```

Example 2: Upload to S3 (Node.js)

```
const { S3Client, PutObjectCommand } = require('@aws-sdk/client-s3');
const fs = require('fs');

const client = new S3Client({ region: 'us-east-1' });

async function uploadFile() {
  const fileContent = fs.readFileSync('./local-file.txt');

  const params = {
    Bucket: 'my-bucket-name',
    Key: 'remote-key.txt',
    Body: fileContent,
    ContentType: 'text/plain'
  };

  await client.send(new PutObjectCommand(params));
  console.log('Upload successful!');
}

uploadFile();
```

AWS SDK for Java (SDK v2)

Java is widely used in enterprise AWS applications, especially with Spring Boot.

Maven dependency (pom.xml):

```
<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>software.amazon.awssdk</groupId>
      <artifactId>bom</artifactId>
      <version>2.21.0</version>
      <type>pom</type>
    </dependency>
  </dependencies>
</dependencyManagement>
```

```

        <scope>import</scope>
      </dependency>
    </dependencies>
  </dependencyManagement>

  <dependencies>
    <dependency>
      <groupId>software.amazon.awssdk</groupId>
      <artifactId>s3</artifactId>
    </dependency>
  </dependencies>

```

Example: List S3 Buckets

```

import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.s3.S3Client;
import software.amazon.awssdk.services.s3.model.Bucket;
import software.amazon.awssdk.services.s3.model.ListBucketsResponse;

public class S3Example {
    public static void main(String[] args) {
        // Uses the same credential provider chain as CLI
        S3Client s3 = S3Client.builder()
            .region(Region.US_EAST_1)
            .build();

        ListBucketsResponse response = s3.listBuckets();
        for (Bucket bucket : response.buckets()) {
            System.out.println("Bucket: " + bucket.name());
        }

        s3.close();
    }
}

```

SDK Credential Resolution

All SDKs use the same credential provider chain as the CLI. The precedence is identical:

1. Explicit credentials in code (not recommended)
2. System environment variables
3. ~/.aws/credentials file
4. ~/.aws/config file
5. ECS container role
6. EC2 instance profile

For applications running on AWS services (Lambda, EC2, ECS, etc.), **always use IAM Roles** — no credential configuration needed in code:

```

# On Lambda or EC2 with an IAM Role – zero configuration needed
import boto3
s3 = boto3.client('s3') # Automatically uses the Role
response = s3.list_buckets()

```

SDK vs. CLI Comparison

Aspect	CLI	SDK
Use case	Shell scripts, one-off tasks, CI/CD	Application code, automation

Aspect	CLI	SDK
Languages	Shell (Bash, PowerShell)	Python, Node.js, Java, Go, .NET, Ruby, PHP, C++
Output	JSON, table, text	Native language objects
Error handling	Exit codes + stderr	Language exceptions
Pagination	Built-in --no-paginate flag	Paginate API
Async support	No	Yes (Node.js, Python async/await, Java CompletableFuture)

Hands-On Labs

Lab 1 — Install and Configure AWS CLI v2

Free Tier Status: Completely Free — IAM operations and STS calls have no cost.

Objective: Install the AWS CLI v2 on your local machine and configure it with a non-root IAM user's credentials.

Prerequisites:

- An AWS account
- An IAM user with programmatic access (created in Chapter 2)
- Admin access or `IAMReadOnlyAccess` policy minimum

Step 1: Install AWS CLI v2

Follow the instructions for your OS in Topic 1. Then verify:

```
aws --version
# Expected: aws-cli/2.x.x Python/3.x.x ...
```

If you see `aws-cli/1.x.x`, you have v1 installed. Uninstall it and follow the v2 instructions.

Step 2: Create an IAM User for CLI Access

Note: If you already have an IAM user with an Access Key from Chapter 2, skip to Step 3. **Do not use your root account credentials for CLI access.**

1. In the AWS Console, navigate to **IAM** → **Users** → **Create user**
2. Username: `cli-user`

3. On the **Permissions** page, select **Attach policies directly**
 4. Attach `AdministratorAccess` (for learning) or a more restrictive policy for production
 5. Click through to **Create user**
 6. Click on the user `cli-user`, then **Security credentials** tab
 7. Click **Create access key**
 8. Use case: **Command Line Interface (CLI)**
 9. Check the confirmation checkbox and click **Next**
 10. Description tag: `My local CLI key`
 11. Click **Create access key**
 12. **IMPORTANT:** Click **Download .csv file** before closing this page. This is the only time you can download the secret key.
-

Step 3: Run the Configuration Wizard

```
aws configure
```

Enter the values from your CSV file:

```
AWS Access Key ID [None]: AKIAIOSFODNN7EXAMPLE
AWS Secret Access Key [None]: wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY
Default region name [None]: us-east-1
Default output format [None]: json
```

Step 4: Verify the Configuration

```
# Verify the identity (confirms credentials work)
aws sts get-caller-identity
```

Expected output:

```
{
  "UserId": "AIDAIOSFODNN7EXAMPLE",
  "Account": "123456789012",
  "Arn": "arn:aws:iam::123456789012:user/cli-user"
}
```

If you see an error like `InvalidClientTokenId`, your Access Key ID is wrong. If you see `SignatureDoesNotMatch`, your Secret Key is wrong.

Step 5: Inspect the Configuration Files

```
# On Linux/macOS
cat ~/.aws/credentials
cat ~/.aws/config
```

```
# On Windows (PowerShell)
type $env:USERPROFILE\.aws\credentials
type $env:USERPROFILE\.aws\config
```

You should see your credentials and configuration stored in INI format.

Step 6: Test the Configuration

```
# This should list zero or more S3 buckets (empty list is fine)
aws s3 ls

# This should show the regions available
aws ec2 describe-regions --output table
```

Cleanup

No AWS resources were created in this lab. Access keys exist in IAM but can be left for use in subsequent labs.

Lab 2 — Run Core CLI Commands

Free Tier Status: Completely Free — describe/list read operations have no cost. Creating resources may have cost implications noted below.

Objective: Practice the three CLI commands from the roadmap: `describe-instances`, `list-buckets`, and `describe-vpcs`. Also explore output formatting and filtering.

Step 1: List S3 Buckets

```
# List all buckets (high-level S3 command)
aws s3 ls

# List objects in a specific bucket (if you have any)
aws s3 ls s3://BUCKET-NAME

# List buckets with creation dates using the API-level command
aws s3api list-buckets

# List just the bucket names
aws s3api list-buckets --query 'Buckets[*].Name' --output text
```

If you have no S3 buckets yet, this will return an empty list — that is fine.

Step 2: Create a Test S3 Bucket (for practice)

Free Tier Note: S3 storage itself is free for the first 5 GB per month. Creating buckets has no charge. Be sure to delete the bucket at the end of this lab.

```
# Bucket names must be globally unique. Replace YOUR_NAME with your name.
```

```
# Use lowercase letters, numbers, and hyphens only.
BUCKET_NAME="aws-cli-practice-YOUR_NAME-$(date +%s)"
echo "Your bucket name: $BUCKET_NAME"

# Create the bucket (in us-east-1, no LocationConstraint needed)
aws s3api create-bucket --bucket $BUCKET_NAME --region us-east-1

# Verify it appears in the list
aws s3 ls | grep $BUCKET_NAME
```

Now list the bucket in different output formats:

```
# JSON output (default)
aws s3api list-buckets --output json

# Table output (human-friendly)
aws s3api list-buckets --output table

# Text output (for scripting)
aws s3api list-buckets --output text

# Filter to show only names created after a date
aws s3api list-buckets --query 'Buckets[?starts_with(Name, `aws-cli`)]'
```

Step 3: Describe EC2 Instances

If you have no EC2 instances running, this command will return an empty Reservations list — that is expected.

```
# Basic describe (all instances in the default region)
aws ec2 describe-instances

# Show as a table
aws ec2 describe-instances --output table

# Extract just InstanceId and State
aws ec2 describe-instances \
  --query 'Reservations[*].Instances[*].\
{ID:InstanceId,Type:InstanceType,State:State.Name,AZ:Placement.AvailabilityZone}\
' \
  --output table

# Filter by instance state (running only)
aws ec2 describe-instances \
  --filters Name=instance-state-name,Values=running \
  --query 'Reservations[*].Instances[*].InstanceId' \
  --output text

# Check a different region
aws ec2 describe-instances --region eu-west-1 --output table
```

Step 4: Describe VPCs

Every AWS account has a **default VPC** in each region. Let's explore it.

```
# List all VPCs in the default region
aws ec2 describe-vpcs
```

```
# Show in table format with key details
aws ec2 describe-vpcs \
  --query 'Vpcs[*].{VpcId:VpcId,CIDR:CidrBlock,Default:IsDefault,State:State}' \
  --output table
```

Expected output (you will see the default VPC):

```
-----+-----+-----+-----+-----+
|                               DescribeVpcs                               |
+-----+-----+-----+-----+-----+
| CIDR | Default | State | VpcId |
+-----+-----+-----+-----+
| 172.31.0.0/16 | True | available | vpc-abc123 |
+-----+-----+-----+-----+-----+
```

Explore the VPC further:

```
# Get subnets in the default VPC
aws ec2 describe-subnets \
  --filters Name=default-for-az,Values=true \
  --query 'Subnets[*].{SubnetId:SubnetId,AZ:AvailabilityZone,CIDR:CidrBlock}' \
  --output table

# List Internet Gateways
aws ec2 describe-internet-gateways --output table

# List security groups
aws ec2 describe-security-groups \
  --query 'SecurityGroups[*].{ID:GroupId,Name:GroupName,Desc:Description}' \
  --output table
```

Step 5: Practice --query Filtering

The --query parameter uses JMESPath. Practice these patterns:

```
# Get the VPC ID of the default VPC
aws ec2 describe-vpcs \
  --filters Name=isDefault,Values=true \
  --query 'Vpcs[0].VpcId' \
  --output text

# Assign to a shell variable
DEFAULT_VPC_ID=$(aws ec2 describe-vpcs \
  --filters Name=isDefault,Values=true \
  --query 'Vpcs[0].VpcId' \
  --output text)

echo "Default VPC ID: $DEFAULT_VPC_ID"

# Use that variable in another command
aws ec2 describe-subnets \
  --filters Name=vpc-id,Values=$DEFAULT_VPC_ID \
  --query 'Subnets[*].{SubnetId:SubnetId,AZ:AvailabilityZone}' \
  --output table
```

Step 6: Explore IAM via CLI

```
# List IAM users
```

```
aws iam list-users --output table

# List IAM groups
aws iam list-groups

# List policies attached to your cli-user
aws iam list-attached-user-policies --user-name cli-user

# Get your current account alias (if set)
aws iam list-account-aliases
```

Cleanup

```
# Delete the test S3 bucket (replace with your actual bucket name)
aws s3 rb s3://$BUCKET_NAME
```

If the bucket has objects:

```
# Force delete (removes all objects first)
aws s3 rb s3://$BUCKET_NAME --force
```

Lab 3 — Use AWS CloudShell

Free Tier Status: Completely Free — CloudShell has no charge.

Objective: Use AWS CloudShell to run commands without any local configuration.

Step 1: Open CloudShell

1. Log into the **AWS Management Console**
 2. In the top navigation bar, click the **CloudShell icon** (>_) — it is to the left of the bell icon
 3. If prompted with a welcome message, click **Close**
 4. Wait for the shell to initialize (a green prompt will appear)
-

Step 2: Verify the Environment

```
# Check CLI version
aws --version

# Check Python version
python3 --version

# Check Node.js version
node --version

# Verify you are authenticated (no configuration needed!)
aws sts get-caller-identity
```

Notice that `aws sts get-caller-identity` works immediately without any `aws`

configure — CloudShell is pre-authenticated with your console session.

Step 3: Run the Same Commands as Lab 2

Repeat the commands from Lab 2 in CloudShell to confirm they produce the same results:

```
# List S3 buckets
aws s3 ls

# Describe VPCs
aws ec2 describe-vpcs \
  --query 'Vpcs[*].{VpcId:VpcId,CIDR:CidrBlock,Default:IsDefault}' \
  --output table

# List IAM users
aws iam list-users --output table
```

Step 4: Use Python with Boto3

```
# Boto3 is pre-installed in CloudShell
python3 << 'EOF'
import boto3

# List S3 buckets
s3 = boto3.client('s3')
response = s3.list_buckets()
print("=== S3 Buckets ===")
for bucket in response.get('Buckets', []):
    print(f"  - {bucket['Name']}")

# Get current identity
sts = boto3.client('sts')
identity = sts.get_caller_identity()
print(f"\n=== Current Identity ===")
print(f"  Account: {identity['Account']}")
print(f"  ARN: {identity['Arn']}")
EOF
```

Step 5: Create and Save a Script in CloudShell

```
# Create a useful script in the persistent home directory
cat > ~/aws-info.sh << 'EOF'
#!/bin/bash
echo "=====
echo " AWS Environment Summary"
echo "=====
echo ""
echo "--- Identity ---"
aws sts get-caller-identity --output table

echo ""
echo "--- S3 Buckets ---"
aws s3 ls

echo ""
echo "--- VPCs ---"
```

```
aws ec2 describe-vpcs \
  --query 'Vpcs[*].{VpcId:VpcId,CIDR:CidrBlock,Default:IsDefault}' \
  --output table

echo ""
echo "---- IAM Users ----"
aws iam list-users \
  --query 'Users[*].{UserName:UserName,Created:CreateDate}' \
  --output table
EOF

chmod +x ~/aws-info.sh

# Run the script
./aws-info.sh
```

Step 6: Switch Regions in CloudShell

```
# The current region is shown in the top-right of the CloudShell window
# You can change it with the region selector or using --region flag

# Check default region
echo $AWS_DEFAULT_REGION

# Run a command in a different region
aws ec2 describe-vpcs --region eu-west-1 --output table

# Temporarily change the default region for the session
export AWS_DEFAULT_REGION=ap-southeast-1
aws ec2 describe-vpcs --output table

# Reset to the original
unset AWS_DEFAULT_REGION
```

Step 7: Upload/Download Files in CloudShell

```
# Create a test file
echo "Hello from CloudShell" > test.txt

# Upload to S3 (replace BUCKET_NAME if you have a bucket)
# aws s3 cp test.txt s3://YOUR_BUCKET_NAME/

# Download a file from S3
# aws s3 cp s3://YOUR_BUCKET_NAME/file.txt .

# Use the CloudShell "Actions" menu to:
# - Upload File: uploads from your local computer to CloudShell
# - Download File: downloads from CloudShell to your local computer
```

Step 8: Explore CloudShell Tabs

You can open multiple CloudShell terminals simultaneously:

1. Click **Actions** → **New tab** (or the + icon)
2. A second shell opens in a new tab

3. Both shells share the same persistent storage directory
 4. Try running `ls ~/` in both tabs — they see the same files
-

Cleanup

No AWS resources were created in this lab. The `~/aws-info.sh` file in CloudShell's persistent storage is harmless and can be kept for future use.

Lab 4 — Configure Multiple Named CLI Profiles

Free Tier Status: Completely Free — profile configuration is local and does not involve AWS API calls (except for the verification step).

Objective: Create multiple named CLI profiles and practice switching between them.

Scenario: You are simulating a developer who works with two AWS accounts — a **sandbox** account for personal experimentation and a **company-dev** account for team development.

Note: In this lab, you will simulate having two accounts using two profiles that point to the **same** account but with different configurations. In a real scenario, each profile would have different `aws_access_key_id` values from different accounts.

Step 1: Create a Second IAM User (Simulating Second Account)

If you want to simulate a second account properly, create another IAM user:

1. In IAM Console, create user `cli-user-2` with `ReadOnlyAccess` policy
2. Create an access key for `cli-user-2` and download the CSV

If you do not want to create a second user, proceed with the same credentials but different region configurations — the concepts are the same.

Step 2: Configure the Default Profile

Verify your default profile is set up (from Lab 1):

```
aws configure list
```

Output should show your `default` profile configuration.

Step 3: Create a "sandbox" Named Profile

```
aws configure --profile sandbox
```

When prompted:

- Access Key ID: Enter your `cli-user` key (or same default key)
 - Secret Access Key: Enter the corresponding secret
 - Default region: `us-east-1`
 - Default output format: `json`
-

Step 4: Create a "dev" Named Profile

```
aws configure --profile dev
```

When prompted:

- Access Key ID: Enter `cli-user-2`'s key (or same key)
 - Secret Access Key: Enter the corresponding secret
 - Default region: `eu-west-1`
 - Default output format: `table`
-

Step 5: Inspect the Configuration Files

```
cat ~/.aws/credentials
```

Expected output:

```
[default]
aws_access_key_id = AKIAIOSFODNN7EXAMPLE
aws_secret_access_key = wJalrXUtnFEMI/K7MDENG/bPxrFiCYEXAMPLEKEY

[sandbox]
aws_access_key_id = AKIAIOSFODNN7EXAMPLE
aws_secret_access_key = wJalrXUtnFEMI/K7MDENG/bPxrFiCYEXAMPLEKEY

[dev]
aws_access_key_id = AKIAI44QH8DHBEXAMPLE
aws_secret_access_key = je7MtGbClwBF/2Zp9Utk/h3yCo8nvbEXAMPLEKEY

cat ~/.aws/config
```

Expected output:

```
[default]
region = us-east-1
output = json

[profile sandbox]
region = us-east-1
output = json

[profile dev]
region = eu-west-1
output = table
```

Step 6: Verify Each Profile

```
# Default profile
aws sts get-caller-identity
echo "Region: $(aws configure get region)"

# Sandbox profile
aws sts get-caller-identity --profile sandbox
echo "Sandbox Region: $(aws configure get region --profile sandbox)"

# Dev profile
aws sts get-caller-identity --profile dev
echo "Dev Region: $(aws configure get region --profile dev)"
```

Notice how running the same command with different profiles connects to different regions (and potentially different accounts).

Step 7: Use `AWS_PROFILE` Environment Variable

```
# Observe the current default
aws ec2 describe-regions \
  --query 'Regions[*].RegionName' \
  --output table

# Switch to "dev" profile for the entire terminal session
export AWS_PROFILE=dev

# All subsequent commands use "dev" profile
aws sts get-caller-identity
aws ec2 describe-vpcs --query 'Vpcs[*].{VpcId:VpcId}' --output table

# Note: the region is now eu-west-1 (as configured in the dev profile)

# Revert to default
unset AWS_PROFILE
```

Step 8: Override Region on the Fly

```
# Use "dev" profile (eu-west-1) but override region to ap-southeast-1
aws ec2 describe-vpcs \
  --profile dev \
  --region ap-southeast-1 \
  --query 'Vpcs[*].{VpcId:VpcId,CIDR:CidrBlock}' \
  --output table
```

The `--region` flag always overrides the profile's configured region for that single command.

Step 9: Create a Role-Assumption Profile

This demonstrates how to configure the CLI to automatically assume an IAM Role:

```
# First, find your account ID
ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)
echo "Account ID: $ACCOUNT_ID"

# Add a role-assumption profile to ~/.aws/config
```

```
cat >> ~/.aws/config << EOF

[profile readonly-role]
role_arn = arn:aws:iam::${ACCOUNT_ID}:role/ReadOnlyRole
source_profile = default
region = us-east-1
EOF
```

Note: The role `ReadOnlyRole` must exist in your account for this to work. If you created the IAM Role from Chapter 2 labs, substitute the actual role ARN. Otherwise, this configuration is valid but will fail to assume a non-existent role — which is expected.

Step 10: List and Clean Up Profiles

```
# List all configured profile names
aws configure list-profiles

# Show full config for a profile
aws configure list --profile dev

# Remove a profile (manual file edit required – no CLI command for this)
# Edit ~/.aws/credentials and ~/.aws/config manually and delete the relevant
sections
```

Step 11: Experiment with Credential Precedence

```
# Show which credentials are currently in use
aws sts get-caller-identity

# Set an environment variable (overrides the credentials file)
export AWS_ACCESS_KEY_ID="FAKE_ACCESS_KEY_FOR_DEMO"
export AWS_SECRET_ACCESS_KEY="fake_secret_key_for_demo"

# Now the CLI tries to use the environment variable (will fail with auth error)
aws sts get-caller-identity
# ERROR: InvalidClientTokenId

# Clear the environment variable – falls back to credentials file
unset AWS_ACCESS_KEY_ID
unset AWS_SECRET_ACCESS_KEY

# Now it works again
aws sts get-caller-identity
```

This demonstrates that environment variables (position 2) override the credentials file (position 3).

Cleanup

No AWS resources were created. The only artifacts are local configuration files, which you can leave in place for future labs.

Real-World Scenarios

Scenario 1: CI/CD Pipeline Authentication

Situation: A development team uses GitHub Actions to deploy a Node.js application to AWS. They need the pipeline to have AWS access.

Wrong approach: Hardcode Access Keys in the GitHub repository.

Right approach:

1. Create a dedicated IAM user `github-deploy-user` with only the required permissions (e.g., `s3:PutObject`, `cloudfront:CreateInvalidation`)
2. Store the Access Key in **GitHub Secrets** (not in the code)
3. Reference secrets in the workflow:

```
# .github/workflows/deploy.yml
- name: Configure AWS credentials
  uses: aws-actions/configure-aws-credentials@v4
  with:
    aws-access-key-id: ${ secrets.AWS_ACCESS_KEY_ID }
    aws-secret-access-key: ${ secrets.AWS_SECRET_ACCESS_KEY }
    aws-region: us-east-1

- name: Deploy to S3
  run: aws s3 sync ./dist s3://my-website-bucket --delete
```

Even better approach: Use GitHub's OIDC integration with AWS IAM Identity Center to get keyless access (no stored secrets at all). But this is beyond the scope of this chapter.

Scenario 2: Multi-Account Developer Workflow

Situation: A developer at a startup has access to three AWS accounts: `sandbox`, `staging`, and `production`. They frequently need to run queries across accounts.

Solution: Named profiles with role assumption:

```
# ~/.aws/config
[default]
region = us-east-1
output = json

[profile sandbox]
role_arn = arn:aws:iam::111111111111:role/DeveloperRole
source_profile = default
region = us-east-1

[profile staging]
role_arn = arn:aws:iam::222222222222:role/DeveloperRole
source_profile = default
region = us-east-1

[profile production]
role_arn = arn:aws:iam::333333333333:role/ReadOnlyRole
source_profile = default
region = us-east-1
mfa_serial = arn:aws:iam::000000000000:mfa/john.doe
```

The developer can now run:

```
# Check sandbox EC2 instances
aws ec2 describe-instances --profile sandbox

# Check staging RDS status
aws rds describe-db-instances --profile staging

# Read-only production query (will prompt for MFA code)
aws s3 ls --profile production
```

Scenario 3: Automated Backup Script

Situation: A small business wants to automatically back up their application database to S3 every night.

Solution: An EC2 instance with an IAM Role (instead of hardcoded keys):

```
#!/bin/bash
# /home/ec2-user/backup.sh
# This script runs on EC2 with an IAM Role that has S3 write access
# NO credentials needed – the Role provides them automatically

DATE=$(date +%Y-%m-%d)
DB_BACKUP="/tmp/database-backup-`${DATE}`.sql"

# Create backup
mysqldump -u root -p mypassword mydatabase > $DB_BACKUP

# Upload to S3 (uses EC2 instance role automatically)
aws s3 cp $DB_BACKUP s3://company-backups/database/

# Clean up
rm $DB_BACKUP

echo "Backup complete: s3://company-backups/database/database-backup-`${DATE}`.sql"
```

The key insight: the script has no `aws configure` step and no credentials. The IAM Role attached to the EC2 instance provides credentials via the Instance Metadata Service automatically.

Key Facts & Exam Cheat Sheet

CLI & SDK – Key Facts for SAA-C03	
CREDENTIAL PROVIDER CHAIN (order matters!):	
1. Command-line options / --profile flag	
2. Environment variables (AWS_ACCESS_KEY_ID, etc.)	
3. ~/.aws/credentials file	
4. ~/.aws/config file	
5. ECS container role	
6. EC2 instance profile (IAM Role on the instance)	
ACCESS KEY PREFIXES:	
AKIA... = Permanent (long-term) key	
ASIA... = Temporary (STS-issued) key	
IAM ROLES vs. ACCESS KEYS:	

- EC2/Lambda/ECS → ALWAYS use IAM Roles
- Local dev → Access Keys (stored in ~/.aws/credentials)
- CI/CD (GitHub, Jenkins) → Access Keys in secrets vault
- Roles auto-rotate. Keys do NOT auto-rotate.

CLOUDSHELL LIMITS:

- 1 GB persistent storage per Region
- Sessions timeout after ~20 min of inactivity
- Runs as your console credentials (no key setup needed)
- Pre-installed: AWS CLI v2, Python3/Boto3, Node.js, git

PROFILE SYNTAX:

```
~/.aws/credentials → [default], [profile-name]
~/.aws/config      → [default], [profile profile-name]
                    (note the "profile " prefix!)
```

CLI OUTPUT FORMATS:

```
json → structured, best for parsing
table → human-readable columns
text → plain text, great for grep/awk in shell scripts
```

BOTO3 KEY CONCEPTS:

```
Client → low-level, direct API mapping
Resource → high-level, object-oriented
Session → isolated credential/config context
```

Certification-Style Practice Questions

Question 1

A developer is running an application on an Amazon EC2 instance. The application needs to read objects from an Amazon S3 bucket. Which approach is the MOST secure and operationally efficient way to grant the application access to S3?

- A) Create an IAM user, generate access keys, and store them in the application configuration file on the EC2 instance.
- B) Create an IAM user, generate access keys, and set them as environment variables in the EC2 instance user data.
- C) Create an IAM role with the required S3 read permissions, and attach the role to the EC2 instance.
- D) Create a root account access key and configure it in the ~/.aws/credentials file on the EC2 instance.

Answer: C

Explanation: An IAM Role attached to an EC2 instance automatically provides rotating temporary credentials via the Instance Metadata Service. The application (including Boto3, CLI, or any SDK) automatically retrieves these credentials without any credential files or environment variables.

Options A, B, and D all involve static long-lived credentials, which are security risks. Option D is especially dangerous as root credentials should never be used for applications.

Question 2

A developer has configured the AWS CLI with default credentials in `~/.aws/credentials`. They have also set the `AWS_ACCESS_KEY_ID` and `AWS_SECRET_ACCESS_KEY` environment variables. They then run a command using `aws s3 ls --profile production`. Which credentials will the CLI use?

- A) The credentials in `~/.aws/credentials` under the `[default]` profile.
- B) The credentials specified by the `AWS_ACCESS_KEY_ID` environment variable.
- C) The credentials stored under the `[production]` profile in `~/.aws/credentials`.
- D) The credentials from the EC2 Instance Metadata Service.

Answer: C

Explanation: The `--profile` flag is the highest priority in the credential provider chain (position 1). It overrides both the environment variables (position 2) and the default profile (position 3). When `--profile production` is specified, the CLI uses the `[production]` profile's credentials regardless of any other configured credentials.

Question 3

A company runs an application on AWS Lambda that needs to access Amazon DynamoDB. The security team requires that no long-lived credentials be stored anywhere. What is the BEST solution?

- A) Create an IAM user for Lambda, generate access keys, and store them in AWS Systems Manager Parameter Store as SecureString parameters.
- B) Create an IAM execution role for the Lambda function with the necessary DynamoDB permissions. Attach the role to the Lambda function.
- C) Store the DynamoDB credentials in the Lambda environment variables encrypted with KMS.
- D) Hardcode the access keys in the Lambda function code and rotate them every 30 days.

Answer: B

Explanation: AWS Lambda, like EC2, supports IAM execution roles. When you attach an execution role to a Lambda function, the function automatically receives temporary, rotating credentials. No credentials need to be stored anywhere — AWS manages the entire credential lifecycle. Options A and C still involve long-lived credentials. Option D is the worst practice possible.

Question 4

A developer is using the AWS CLI on their local workstation. They want all CLI commands in a specific terminal session to use a profile named "staging" without adding `--profile staging` to every command. What is the correct approach?

- A) Edit `~/.aws/credentials` and rename `[staging]` to `[default]`.
- B) Run `aws configure --profile default` and re-enter the staging credentials.
- C) Set the environment variable `export AWS_PROFILE=staging`.
- D) Set the environment variable `export AWS_DEFAULT_PROFILE=staging`.

Answer: C

Explanation: The `AWS_PROFILE` environment variable overrides the default profile for all subsequent CLI commands in that terminal session. It is temporary (does not persist across sessions) and does not modify any files. Option A permanently changes the default profile. Option B reconfigures the default permanently. Option D uses a non-existent environment variable name — the correct variable is `AWS_PROFILE`.

Question 5

Which of the following statements about AWS CloudShell is CORRECT?

- A) CloudShell requires you to configure AWS credentials using `aws configure` before use.
- B) Files stored in CloudShell are deleted when the browser session ends.
- C) CloudShell provides 1 GB of persistent storage per AWS Region.
- D) CloudShell charges \$0.01 per minute of active use.

Answer: C

Explanation: CloudShell provides 1 GB of persistent storage per Region. Files in the home directory survive session restarts, browser closures, and even days of inactivity. CloudShell is pre-authenticated with your console session (no `aws configure` needed), storage is persistent (not deleted on session end), and CloudShell is completely free.

Question 6

A developer accidentally committed an AWS Access Key to a public GitHub repository. What should be the IMMEDIATE first response?

- A) Change the IAM user's password.
- B) Delete and recreate the IAM user.
- C) Immediately deactivate or delete the exposed Access Key in IAM.
- D) Rotate the Access Key by creating a new one, then delete the old one.

Answer: C

Explanation: The exposed key must be deactivated or deleted IMMEDIATELY as the first action. Rotating (Option D) creates a new key but the old one remains active during the transition, leaving the compromised key exploitable. After deactivating/deleting the key, you should also review

CloudTrail logs to assess what actions may have been taken with the key. Creating a new key comes after the compromised one is disabled.

Question 7

An application running on an EC2 instance uses Boto3 to interact with AWS services. The instance has an IAM role attached with S3 full access. The EC2 instance also has `AWS_ACCESS_KEY_ID` set as an environment variable for a different, less privileged IAM user. What permissions will the Boto3 client have?

- A) The permissions of the IAM role attached to the EC2 instance.
- B) The permissions of both the IAM role and the IAM user combined.
- C) The permissions of the IAM user associated with the environment variable credentials.
- D) No permissions — conflicting credentials cause an authentication error.

Answer: C

Explanation: The credential provider chain checks environment variables (position 2) BEFORE the EC2 instance profile (position 6). Since `AWS_ACCESS_KEY_ID` is set as an environment variable, Boto3 uses those credentials and never checks the instance role. This is a common gotcha — environment variable credentials on an EC2 instance silently override the IAM Role, potentially reducing permissions (or increasing them, which is a security risk). This is why you should never set AWS credential environment variables on EC2 instances.

Question 8

A developer is configuring a named profile in the AWS CLI to automatically assume an IAM role in a different AWS account. Which two files are involved in this configuration?

- A) `~/.aws/credentials` and `/etc/aws/config`
- B) `~/.aws/credentials` and `~/.aws/config`
- C) `~/.aws/config` only
- D) `~/.aws/role-config` and `~/.aws/credentials`

Answer: B

Explanation: Role assumption profiles use both `~/.aws/credentials` (for the `source_profile`'s access keys) and `~/.aws/config` (for the `[profile name]` section containing `role_arn` and `source_profile`). The credential itself is in `~/.aws/credentials`, and the role ARN configuration is in `~/.aws/config`. Only `~/.aws/config` is technically required if using a named profile that itself has `role_arn`, but the `source_profile` referenced must have credentials in one of the two files.

Interview Questions

Q1: What is the credential provider chain in AWS, and why does it matter?

Answer: The credential provider chain is the ordered sequence of locations the AWS CLI and SDKs check to find credentials. The order is: (1) command-line options/profile flag, (2) environment variables, (3) `~/.aws/credentials` file, (4) `~/.aws/config` file, (5) ECS container role, (6) EC2 instance profile.

It matters because it determines which credentials are actually used when multiple sources are configured, which directly affects security. For example, if a developer accidentally sets credential environment variables on an EC2 instance, these override the IAM Role (best practice), potentially reducing security. Understanding the chain helps debug authentication issues and design secure architectures.

Q2: Why should you use IAM Roles instead of Access Keys for EC2 instances?

Answer:

- **Automatic rotation:** Role credentials automatically expire and rotate, reducing the blast radius of credential theft.
 - **No storage required:** Credentials are never stored in files or environment variables — they are delivered in-memory via the Instance Metadata Service on demand.
 - **No manual management:** No need to create, rotate, distribute, or track credentials.
 - **Principle of least privilege:** Roles make it easy to grant only the permissions an instance needs.
 - **Auditability:** CloudTrail events show which role (and therefore which instance) took an action.
-

Q3: What is the difference between `aws configure` and manually editing `~/.aws/credentials`?

Answer: Both achieve the same result. `aws configure` is an interactive wizard that writes values to the files automatically, validates input, and is user-friendly. Manually editing the files gives you more control, allows bulk edits, and is necessary for options not exposed by the wizard (like `duration_seconds` or `mfa_serial`). In CI/CD pipelines, the files are often written programmatically rather than using `aws configure`.

Q4: What is the difference between `boto3.client()` and `boto3.resource()`?

Answer:

- `boto3.client()` provides a **low-level** interface that maps directly to the AWS API operations. It returns dictionaries of raw API responses. It has full coverage of all AWS API features.
- `boto3.resource()` provides a **high-level**, object-oriented interface with abstractions (e.g., `s3.Bucket('name').upload_file()`). It is more Pythonic and easier to read, but not all services have Resource abstraction.

Use `client()` when you need fine-grained control or the Resource doesn't exist for a service. Use `resource()` for common CRUD operations on supported services (S3, EC2, DynamoDB, etc.).

Q5: How would you verify that an EC2 instance's IAM Role is correctly providing credentials to an application?

Answer:

```
# From inside the EC2 instance:
# 1. Check the instance metadata for the role name
curl http://169.254.169.254/latest/meta-data/iam/security-credentials/

# 2. Get the temporary credentials for that role
curl http://169.254.169.254/latest/meta-data/iam/security-credentials/ROLE_NAME

# 3. Use the CLI to verify identity (note: ASIA prefix confirms temporary creds)
aws sts get-caller-identity

# 4. Test the specific action the application needs
aws s3 ls # or whatever action is needed
```

You should see an assumed-role ARN in the `get-caller-identity` response, confirming the instance is using the role's credentials.

Q6: How does the AWS CLI handle paginated API responses?

Answer: Many AWS APIs return results in pages (to prevent overwhelming responses for large datasets). The CLI handles this in two ways:

Automatic pagination (default for most high-level commands): The CLI automatically makes multiple API calls and aggregates all results:

```
aws s3api list-objects-v2 --bucket my-large-bucket
# Returns ALL objects, making multiple API calls internally
```

Manual pagination (using `--max-items` and `--starting-token`):

```
aws ec2 describe-instances --max-items 10
# Returns first 10 and a NextToken
aws ec2 describe-instances --max-items 10 --starting-token TOKEN
```

```
# Returns the next 10
```

You can disable autopagination with `--no-paginate` for better performance when you only need the first page.

Q7: A developer says "I set `AWS_ACCESS_KEY_ID` but my commands still use the wrong credentials." What would you check?

Answer:

1. Verify the variable is exported (not just set): `echo $AWS_ACCESS_KEY_ID`
 2. Check for typos: is it `AWS_ACCESS_KEY_ID` (not `AWS_ACCESS_KEY`)?
 3. Verify `AWS_SECRET_ACCESS_KEY` is also set
 4. Check if `--profile` is being used in the command (overrides env vars)
 5. Run `aws sts get-caller-identity` and check the ARN
 6. Check if there is a wrapper script or alias that adds `--profile` automatically
 7. If using temporary credentials, verify `AWS_SESSION_TOKEN` is also set
-

Q8: What is AWS CloudShell and when would you recommend it over a local CLI installation?

Answer: AWS CloudShell is a browser-based, pre-authenticated shell environment embedded in the AWS Console. It comes with AWS CLI v2, Python/Boto3, Node.js, git, and other tools pre-installed, with 1 GB of persistent storage per Region.

Recommend CloudShell when:

- Working on a shared/public computer where you cannot install software
- Needing a quick command without local setup (e.g., during an incident from a different machine)
- Teaching/demonstrating AWS to others without requiring them to install anything
- Working from a restricted corporate device
- Needing a consistent environment across team members

Recommend local CLI when:

- Running long-duration scripts
- Needing to work offline
- CI/CD pipeline integration
- Requiring custom software not available in CloudShell
- High-volume data transfers

Chapter Summary

In this chapter, you have learned everything you need to know about programmatically accessing AWS for both the SAA-C03 exam and real-world use:

AWS CLI v2 is the command-line tool for managing AWS from your terminal. Install it via the official installer (never `pip install awscli` for v2), configure it with `aws configure`, and use the `--query`, `--output`, `--region`, and `--profile` flags to control output and targeting.

AWS CloudShell is a free, pre-authenticated browser-based terminal inside the AWS Console. It requires zero setup and is perfect for quick commands and training, but is not suitable for long-running automation.

Access Keys vs. IAM Roles is a critical security topic. Access Keys are permanent credentials that must be managed, rotated, and protected carefully. IAM Roles provide automatic, rotating, temporary credentials. For any AWS compute service (EC2, Lambda, ECS), always use IAM Roles — never store Access Keys on AWS resources.

The credential provider chain defines the exact order AWS checks for credentials: command-line → environment variables → credentials file → config file → container role → EC2 instance profile. The highest-priority source wins. Understanding this chain is essential for debugging authentication issues and designing secure systems.

Named profiles allow you to store and switch between configurations for multiple AWS accounts or roles. Use `aws configure --profile NAME` to create them, `--profile NAME` to use them per-command, or `export AWS_PROFILE=NAME` to use them for an entire terminal session.

AWS SDKs (Boto3, Node.js SDK v3, Java SDK v2) use the same credential chain and bring AWS API access into your application code. Use `boto3.client()` for full API access and `boto3.resource()` for a higher-level, more Pythonic interface.

What's Next

In **Chapter 4 — Billing, Pricing & Support**, you will learn how AWS charges for services, how to control and forecast costs, and how to set up billing alerts to avoid surprises. Understanding the cost model is essential for the 20% of SAA-C03 questions focused on cost-optimised architectures.

Chapter 3 Complete — AWS CLI, SDK & Access Methods Phase 1: Cloud Foundations | Unit U02
| Difficulty: Beginner